SPEC 2111

Deep Learning and Neural Networks

Dr. Öznur Alkan

Week - 6

2026/2

Course Roadmap

PART-1: Introduction, Foundations & Essentials

- *Week 1 – Introduction & Course Overview*
- *Week 2 – Linear Regression*
  - *Regression concepts, gradient descent, loss functions*
- *Week 3 – Logistic Regression*
  - *Classification, sigmoid, cross-entropy loss*

PART-2: Neural Networks, Deep Learning and Training

- *Week 4 – Neural Network Essentials*
  - *Neurons, layers, forward propagation, activation functions*
- *Week 5 – Training, Refining Backpropagation and Deep Neural Networks (DNNs), Backpropagation, chain rule, gradient descent optimization*
- *Week 6 – Improving Performance, Loss and Activation Functions, Preventing Overfitting, Optimizers*

PART-3: Specialized Architectures

- *Week 7 – Convolutional Neural Networks (CNNs)*
  - *Convolutions, pooling, image applications*
  - *Assessment-1*
- *Week 8 – Representations & Transfer Learning (and Test)*
  - *Feature learning, pre-trained models, fine-tuning*
- *Week 9 – Recurrent Neural Networks & LSTM*
  - *RNNs, LSTM/GRU, sequence modeling*
- *Week 10 – Attention & Transformers*
  - *Attention mechanism, transformer architecture, NLP applications*

PART-4: Reinforcement Learning & Projects

- *Week 11 – Reinforcement Learning*
  - *Assessment-2*
- *Week 12 - Project Presentations & Discussions (Project Due)*

Week-5 / Agenda

Theme: Improving Performance

- Theory
  - Recap - Week 5 on Training Neural Network Essentials
  - Loss functions
  - Alternative Activation Functions
  - Preventing Overfitting
  - Different Optimizers
  - Mock Quiz - not graded
- Lab Work

What is Training a Neural Network Mean?

RECAP

We want the network to **learn a function** that maps inputs x to outputs y as accurately as possible.

$$\hat{y} = f(x; W, b)$$

- The network has **parameters** (weights and biases) **θ={w,b}**
  - These are the “knobs” we can adjust.

We are learning:

- **Which weights and biases produce outputs closest to the true labels y**

Network Parameters?

- **Weights (w)** → control how strongly one neuron influences another
- **Biases (b)** → allow neurons to shift the activation function
- Together, weights + biases define **how the network processes inputs**

Learning = finding the parameter values that **minimize the loss function** over our training data.

- Mathematically:
  Find θ such that J(θ) (cost) is minimized

Every training example gives feedback on how to adjust these parameters.

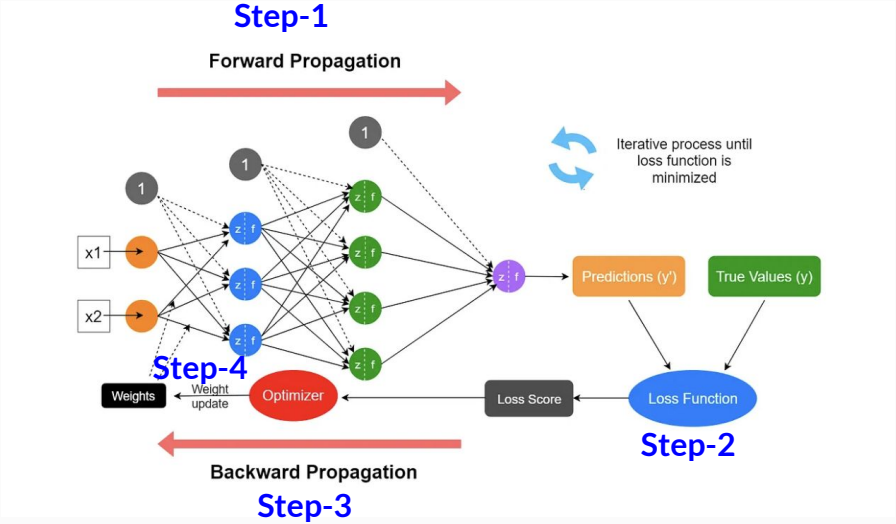
Training a Neural Network

RECAP

Training of a neural network is an iterative process in which the calculations are carried out **forward** and **backward** ***through each layer*** in the network *until the loss function is minimized*.

Training a neural network involves repeating 4 steps:

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. Backward Pass (**Backpropagation**) → Compute gradients
4. Update Weights → Reduce error



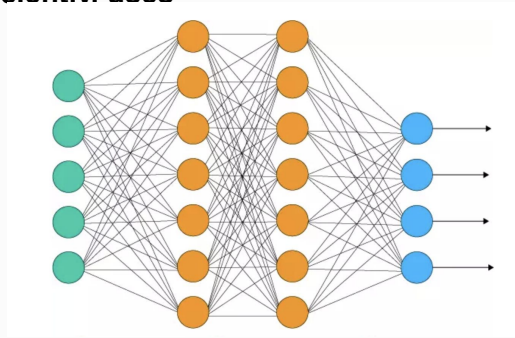
Repeat until convergence.

Backpropagation = **gradient computation algorithm** for neural networks.

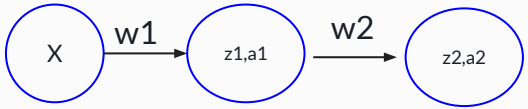
Backpropagation

Gradient Calculation is Tricky is NN

Backpropagation = a systematic way to **calculate gradients for all network parameters efficientlv. uses dynamic programming under the hood to reuse intermediate results**



- Why we need it:
  a. Directly computing derivatives for deep networks is messy
  b. Backprop applies the **chain rule layer by layer**
  c. Computes exactly how each weight/bias affects the final loss
- Once we have gradients → we can **update the parameters** with gradient descent



the chain is exactly:

$$w^{(1)} \to z^{(1)} \to a^{(1)} \to z^{(2)} \to a^{(2)} \to E$$

Applying the Chain Rule

$$\frac{\partial E}{\partial w^{(1)}} = \frac{\partial E}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial w^{(1)}}$$

Calculate Delta / Error Signal in Backpropagation for each node

And they are propagated backwards layer by layer

RECAP

For a weight connecting:

Source neuron i → Destination neuron j

$$\Delta w_{ij} \propto a_i \cdot \delta_j$$

$$\frac{\partial E_{total}}{\partial w_{ij}} = \delta_j \cdot a_i$$

$$\delta_j = \frac{\partial E_{total}}{\partial z_j}$$

Activation signal/input: How much does an input neuron affect the output error?

Error signal: The error signal of a neuron — how responsible this neuron is for the final loss.

- You **must calculate δ for every neuron** during backpropagation, because each one affects how we update the weights feeding into it.
- Once you have δ for a neuron, **updating all incoming weights is just δ × input activations**.

Where Is Loss Used?

The loss function:

- Quantifies error
- Provides gradients (via backprop)
- Guides weight updates

$$\text{Loss} \rightarrow \text{Gradients} \rightarrow \text{Weight Updates}$$

During training:

1. Forward pass → compute prediction
2. **Compute loss → compare prediction vs target**
3. Backpropagation → compute gradients of loss
4. Gradient descent → update weights

Best Combinations: Activation + Loss Functions

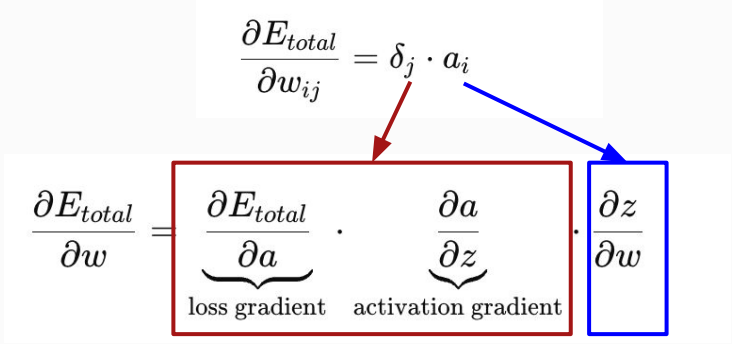
Problem	Output Activation	Loss Function
Regression	Linear	MSE
Binary classification	Sigmoid	Binary Cross-Entropy
Multi-class classification	Softmax	Cross-Entropy

- Good combinations simplify backpropagation.
- Example: Sigmoid + Cross-Entropy gives:

$$\delta = a - y$$

- instead of a complicated expression. This makes training faster and more stable.

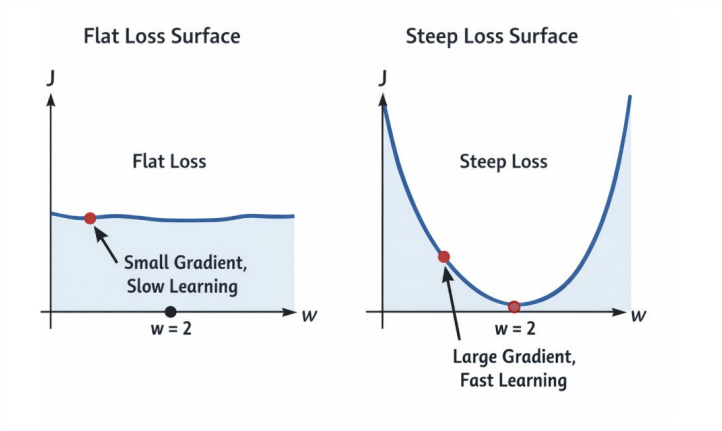
Loss and Activation Functions and Gradient Dynamics



- Choosing activation and loss immediately shapes training dynamics, since they directly impact how network updates parameters!

Flat vs Steep Surfaces

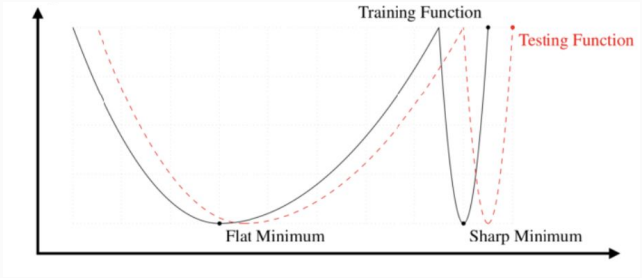
controls **how fast we learn**.



- If loss surface is flat
  - small gradients → tiny weight updates.
  - learning is slow.
- If loss surface is steep
  - large gradients → large weight updates
  - learning is fast but can be unstable.

Curvature (Smooth vs Sharp Valleys)

controls **stability and sensitivity to learning rate**.



- sharp surface
  - region is narrow and sensitive,
  - step size you can safely take is small.
  - unstable or sensitive training
- Low curvature
  - stable but possibly slow

Where Do Vanishing / Exploding Gradients Come From?

$$\delta = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial a_{prev}} \cdot \dots$$

- Backpropagation multiplies many terms together:
- In deep networks, gradients involve products of: activation derivatives, weights layer after layer
- Exponential shrinking gradients happen due to repeated multiplication of weights and activation derivatives across deep layers.
- Vanishing Gradients: Gradients shrink toward zero
- Exploding Gradients: Weights grow extremely large

Assume:

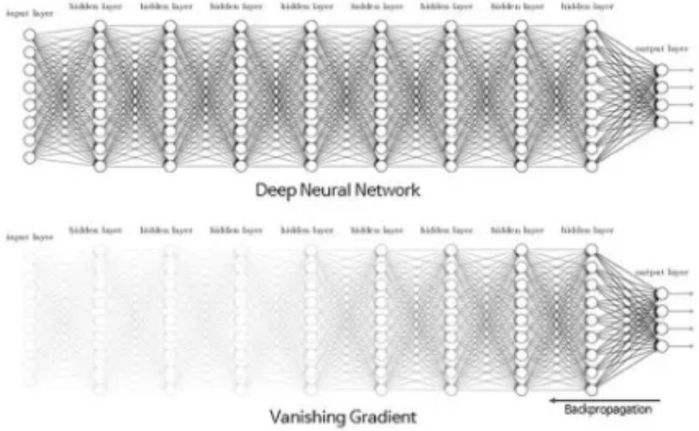
- 10 hidden layers, using sigmoid activation, average sigmoid derivative ≈ 0.1 (this is common)

So during backprop we multiply something like:

$0.1 \times 0.1 \times 0.1 \times \dots$(10 times) = $0.1^{10} = 0.0000000001$

$$\frac{\partial E_{total}}{\partial w^{(1)}} \approx 10^{-10}$$

- That is practically zero.
- The first layer barely updates.



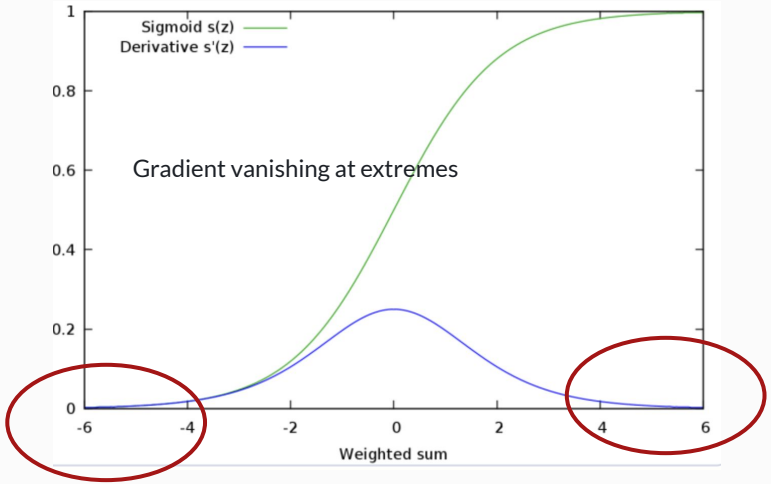
If we use MSE for classification with Sigmoid Activation

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

- Sigmoid output: $a = \sigma(z)$
- MSE loss: $E_{total} = \frac{1}{2}(y-a)^2$

Gradient wrt pre-activation z (error signal):

$$\delta = \frac{\partial E_{total}}{\partial z} = (a-y) \cdot a \cdot (1-a)$$



- When target=1, prediction(a)=0
  - error = ½(1-0)^{2} = =0.5, □ = (0-1) * 1 * (1-1) =~0
  - gradient becomes 0, you cannot update the parameter even when there is a big prediction error!
- **With MSE + sigmoid, gradients vanish when the prediction is near 0 or 1, even if the error is large.**
- This causes **tiny weight updates** and **slow learning**, which is why MSE is generally **not preferred for classification**.

Instead if we use Cross Entropy Loss with Sigmoid

Gradient for cross entropy with sigmoid $\delta = a - y$

For **sigmoid + cross-entropy**, gradient = a−y → **does NOT vanish**, even if prediction is near 0 or 1.

KEY TAKEAWAY

- Cross-entropy gradient remains proportional to **error**, not shrunk by derivative
- Learning is much faster, even in extreme predictions

Softmax Activation Function

- softmax activation function is commonly used in output layers for multi-class classification
- it **converts raw scores into probabilities**
- provides a smooth, differentiable approximation to the argmax function
- **Softmax outputs probabilities that sum to 1.**
- Smooth and differentiable → gradients can be computed for backpropagation.
- Approximates the argmax (max) function in a differentiable way.

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

where:

- e^{z_j}: Exponentiation of the input value.
- $\sum_{j=1}^{n} e^{z_j}$: Sum of all exponentiated values to normalize outputs.

Each output $\sigma(z_i)$ represents the probability of class i.

Key Takeaways: Loss Function + Activation

Loss + Activation Together Control Learning

- The combination determines the **strength and stability of gradients**
- Example:
  - MSE + sigmoid → gradient shrinks near 0/1 → slow learning
  - Cross entropy + sigmoid → gradient remains strong → faster convergence

Choose the Right Loss for Your Problem

- Regression → MSE or variants
- Binary classification → Sigmoid + Cross Entropy
- Multi-class → Softmax + Cross Entropy

Consequences of Choosing Wrong

- Gradients may vanish or explode
- Updates too small or unstable → **model may never converge**
- Training may be extremely slow or fail

OTHER ACTIVATION FUNCTIONS

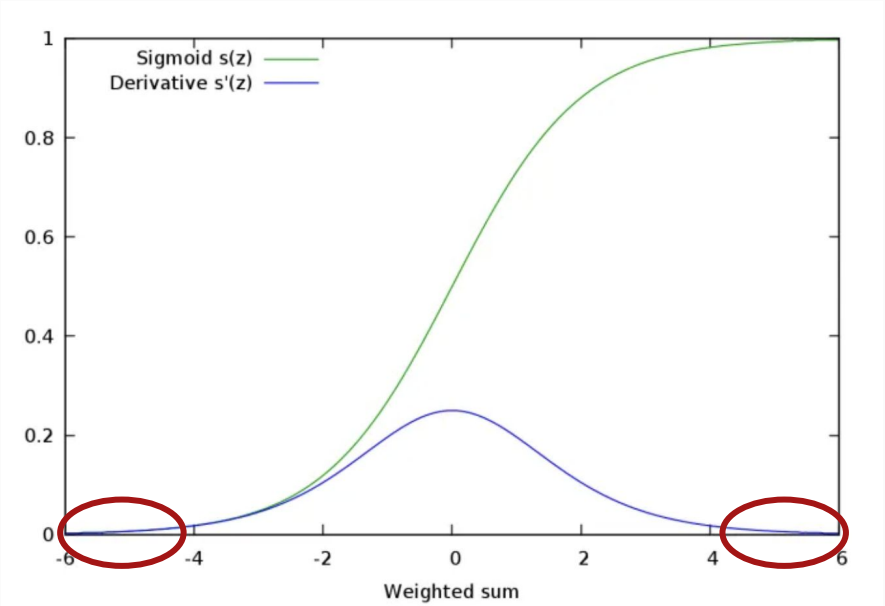
- We saw Sigmoid
- We will check
  - Hyperbolic Tangent (tanh)
  - RELU

Other Activation Functions

$$\frac{\partial E_{total}}{\partial w} = \underbrace{\frac{\partial E_{total}}{\partial a}}_{\text{loss gradient}} \cdot \underbrace{\frac{\partial a}{\partial z}}_{\text{activation gradient}} \cdot \frac{\partial z}{\partial w}$$

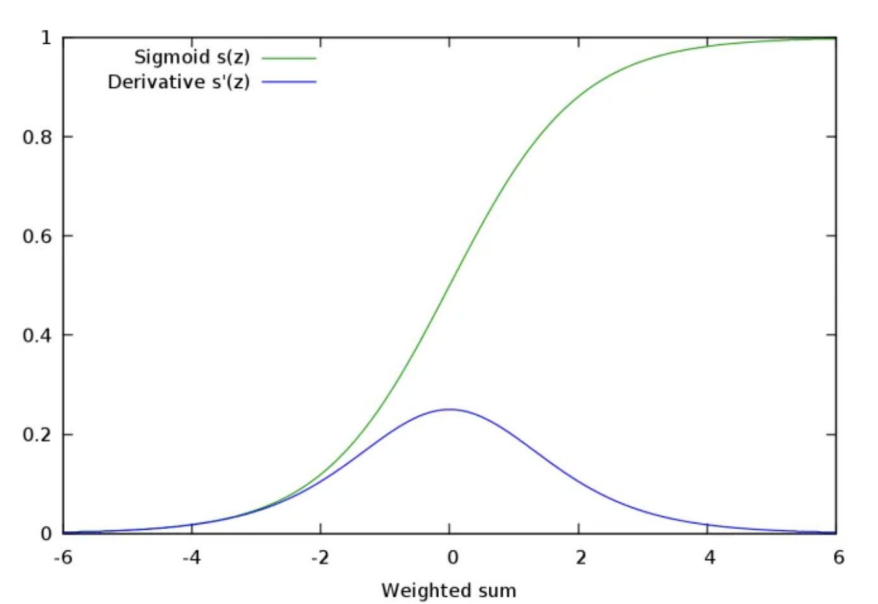
- loss functions are not the only way to improve performance.
- magnitude of the activation gradient effects how fast a neuron learns.
  - If it is small → learning slows.
  - If it is large → learning speeds up.
- sign determines the direction of the update.
  - Zero-centered activations help make gradient directions more balanced.
  - Weight updates naturally move in both positive and negative directions

Problem with Sigmoid



- Derivative becomes very small when z → large positive or large negative
- Not 0-centered
  - Sigmoid's derivative is always positive

Problem with Sigmoid



- For output layer, using sigmoid is ok
- When Sigmoid used in Hidden Layers
  - During backpropagation, activation gradients are multiplied layer by layer
  - Max activation derivative with sigmoid is 0.25!
  - Derivative can become **very small near 0 and 1** (saturated regions)
    - Small derivatives → small gradients
    - Repeated multiplication of small values → **vanishing gradients**
  - Early layers receive almost no update
  - Leads to:
    - Slow convergence
    - Poor learning in deep networks

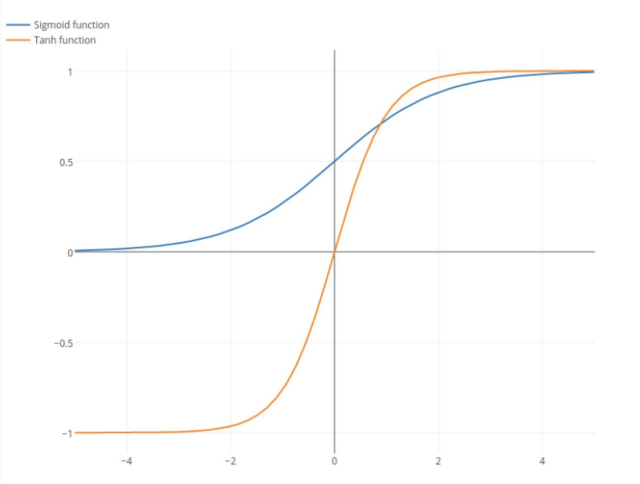
- **Problem occurs in deep hidden layers**
- **Not a problem in the output layer**
- Sigmoid is fine for **binary classification output with cross-entropy**

Applying the Chain Rule

$$\frac{\partial E}{\partial w^{(1)}} = \frac{\partial E}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial w^{(1)}}$$

Hyperbolic Tangent (tanh) Activation largely solves this problem

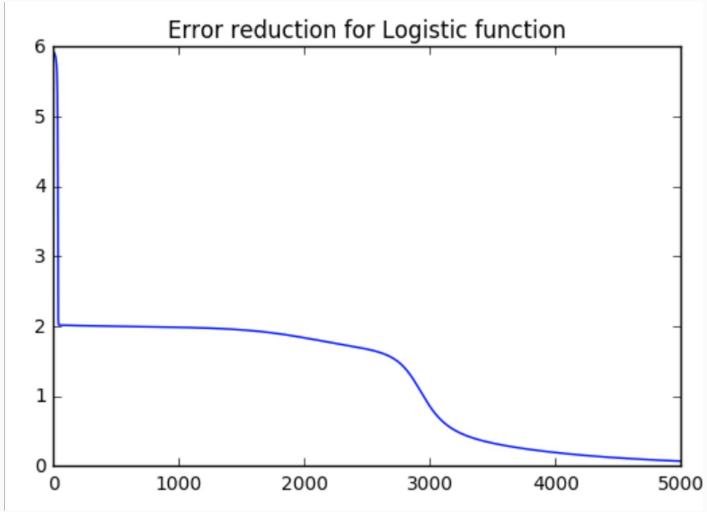
$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad g'(z) = 1 - \tanh^2(z)$$

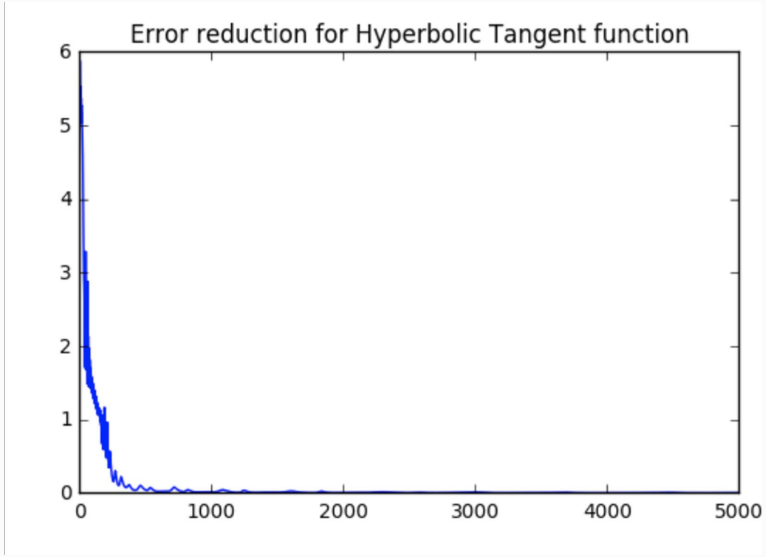


Properties

- Zero-centered, generates both positive and negative gradients
- Outputs values between [-1, 1]
- Larger derivatives than sigmoid near origin
- Often used in hidden layers of neural networks

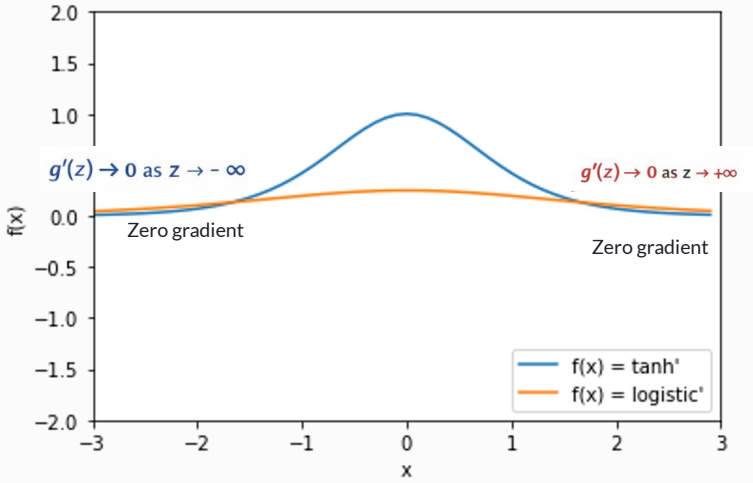
Example of Error Reduction in Iterations





- We trained the same network with MSE loss using sigmoid and tanh activations.
- Tanh converged faster and reached lower error earlier due to its zero-centered outputs and steeper gradients, even though its output range (-1 to 1) does not exactly match our target labels (0/1).

Tanh does not eliminate vanishing gradients entirely!



- helps deeper networks train more effectively than sigmoid (being not 0-centered and max =1 >> 0.25)
- At extremes, saturation of gradients still exists

Small |z| (center of tanh): gradient ≈ 1 → good learning

Large |z| (saturation): gradient ≈ 0 → vanishing gradient

Activation	Derivative max	Output range	Gradient update size
Sigmoid	0.25	(0,1)	small → vanishing
Tanh	1	(-1,1)	larger → less vanishing

tanh was widely used in classic deep networks and RNNs before ReLU became standard.

Beyond Tanh: Can We Do Even Better?

Tanh is better than sigmoid:

- But still saturates at large $|z| \rightarrow$ vanishing gradient in deep layers

Can we design an activation that avoids saturation and keeps gradients large enough for most inputs?

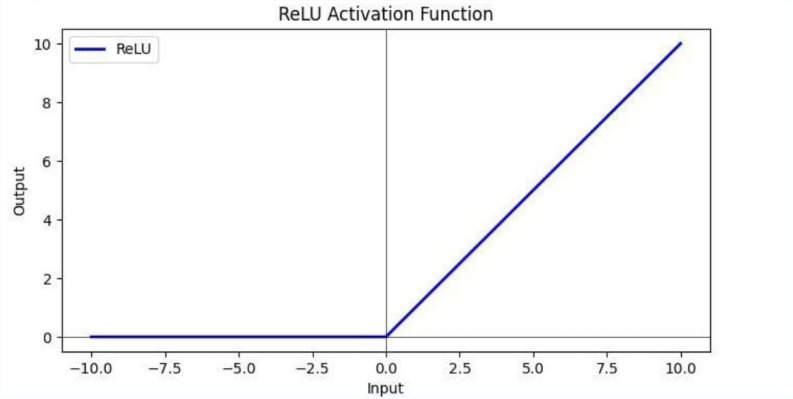
ReLU (Rectified Linear Unit)

- it has constant gradient for positive inputs and mitigates vanishing gradients.

Rectified Linear Unit (ReLU)

$$\mathrm{ReLU}(z) = \max(0, z)$$

RELU for values ranging from -10 to 10



- piecewise linear function that converts its inputs to non-negative outputs.
- When we pass a positive value, say 7.2, to ReLU function, it returns 7.2.
- On the other hand, if we pass a negative value, say -5.5, to ReLU, we get zero as the output.

Gradient of ReLU

ReLU Definition

$$g(z) = \max(0, z)$$

or equivalently

$$g(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

Derivative of ReLU (Mathematically)

$$g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ \text{undefined} & \text{if } z = 0 \\ 1 & \text{if } z > 0 \end{cases}$$

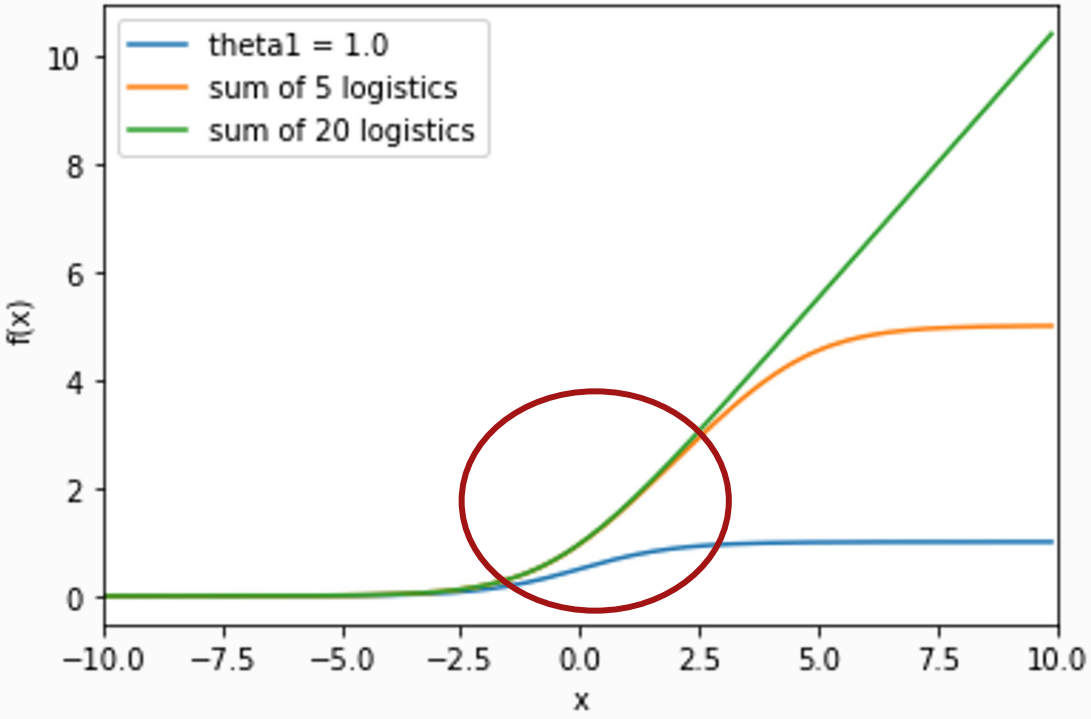
In implementations, we simply define: $g'(0) = 0$

$$g'(z) = \begin{cases} 0 & z \leq 0 \\ 1 & z > 0 \end{cases}$$

ReLU looks very different from tanh or logistic (sigmoid):

- It is not smooth
- It is not differentiable at z=0
- Its derivative is discontinuous

Stacked Sigmoids



ReLU looks strange because:

- It's not smooth at 0
- It has a sharp corner
- It's very simple

But mathematically:

- ReLU can be seen as the limit of smooth nonlinear functions as their slope becomes very steep.
- it's just what you get if you make smooth curves like the sigmoid very steep, turning the soft S-shape into a sharp corner at zero.

Why ReLU works better in deep networks?

Compared to sigmoid/tanh:

- While tanh saturates at both extremes → small gradient → vanishing
- ReLU does not saturate for positive inputs → gradient = 1 → no vanishing for positive values
- ReLU is simpler and computationally efficient

$$g'(z) = \begin{cases} 0 & z \leq 0 \\ 1 & z > 0 \end{cases}$$

ReLU also have sparse network effect

$$g'(z) = \begin{cases} 0 & z \leq 0 \\ 1 & z > 0 \end{cases}$$

For ReLU:

- If z<0, neuron outputs: 0
- in a randomly initialized network:
  - Roughly half of the inputs to a neuron are negative
  - So roughly half of neurons output 0
- That is sparsity!
  - Only a subset of neurons are active
  - The rest are completely shut off

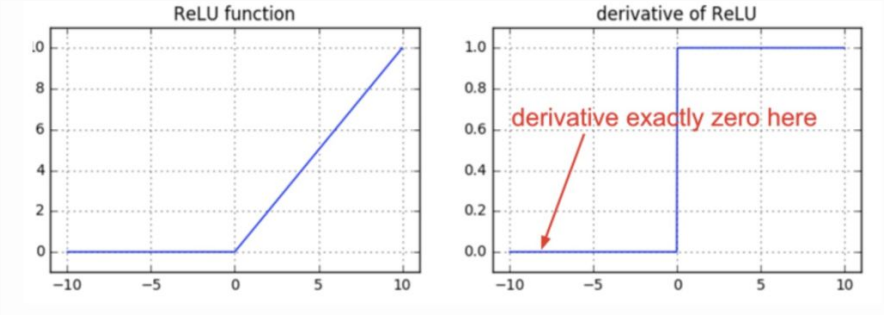
- Sigmoid/Tanh: Every neuron is always slightly active.
- ReLU: A neuron is either fully active or completely silent.
  - ReLU creates a network where only “relevant” neurons fire for a specific input

Advantages of Sparse Network Effect

- Computational efficiency:
  - fewer multiplications, less computation
  - faster training and inference
- Feature Specialization:
  - different neurons respond to different patterns
    i. ex: some neurons activate only for edges, some only for curves, some only for specific objects
  - this encourages specializations
- Implicit Regularization:
  - Since not all neurons are active all the time: model is less likely to overfit, it behaves like dropout

Problem with ReLu: Dying ReLU

- Dying ReLU problem:
  - For **negative inputs**, output = 0 and gradient = 0
  - If a neuron **always gets negative input**, its output stays 0 → gradient is always 0 → neuron **stops learning**



$$\mathrm{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

To solve this: Leaky ReLU Activation Function

Definition:

$$g(z) = \begin{cases} z & \text{if } z \geq 0 \\ \alpha z & \text{if } z < 0 \end{cases}$$

- α is a small constant (e.g., 0.01)
- Allows a **small gradient** for negative inputs, so neurons can still learn

---

Derivative

$$g'(z) = \begin{cases} 1 & z \geq 0 \\ \alpha & z < 0 \end{cases}$$

- Avoids dying neurons problem of standard ReLU
- Still computationally efficient
- Use Leaky ReLU when standard ReLU causes some neurons to “die”
  - Track activations, if a large fraction (>50%) is always 0, you might be experiencing dying ReLUs.

Guidance on Activation Functions

Intuition:

- Hidden layers → want **fast gradients, zero-centered if possible** → **ReLU or tanh**
- Output layer → must **match the target** → **sigmoid, softmax, or linear**

Hidden layers

- Avoid **sigmoid** because of vanishing gradients
- Use **ReLU / Leaky ReLU** for deep networks
- Consider **tanh** for RNNs or small networks
    - However for modern deep networks which are not small, we rarely use **tanh** or **sigmoid** hidden layers unless there's a specific reason

Output layer

Task	Output Activation
Binary classification	Sigmoid
Multi-class classification	Softmax
Regression (unbounded)	Linear
Regression (bounded)	Sigmoid / tanh

Initialization in Deep Networks

In general, when you Initialize to

- Too Large Weights
  - activations can explode
  - training becomes unstable
- Too Small Weights
  - activations shrink toward zero
  - learning slows dramatically
- Many Negative Inputs
  - Many neurons output 0
  - Gradients become 0
  - Neurons may permanently stop learning→ called the “dead ReLU” problem.

initialization is especially important for ReLU networks.

- ReLU zeros out all negative inputs.
- If initialization pushes most inputs negative:
  - A large portion of the network becomes inactive
  - Gradients do not flow through those neurons

Initialization in Deep Networks

Initialization: **giving the network a running start**, not a permanent guarantee.

- Goal is to avoid disastrous scale problems at the start
- training can then fine-tune the weights naturally

Xavier (Glorot) Initialization

- Scale weights based on number of input and output units in a layer to maintain approximately constant activation variance across layers.

Purpose:

- Prevent vanishing/exploding activations
- Stabilize signal propagation in deep networks

Recommended for:

- Sigmoid, Tanh activations

He (Kaiming) Initialization

increases weight variance to compensate for the fact that ReLU sets roughly half of activations to zero, which would otherwise reduce signal variance across layers.

Benefits:

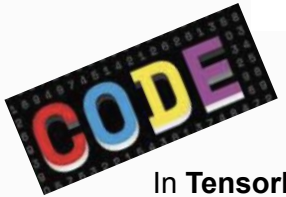
- Maintains stable forward signal propagation
- Reduces risk of dead neurons

Recommended for:

- ReLU, Leaky ReLU, Other ReLU-based activations

Initialization in Deep Networks

Activation Function	Recommended Initialization
Sigmoid / Tanh	Xavier (Glorot)
ReLU / Variants	He (Kaiming)



In **TensorFlow / Keras**, you specify the initialization inside the layer definition using the argument:

```
kernel_initializer=
```

(For biases, use `bias_initializer=`.)

```python
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(
        128,
        activation='tanh',
        kernel_initializer='glorot_uniform'   # Xavier initialization
    )
])
```

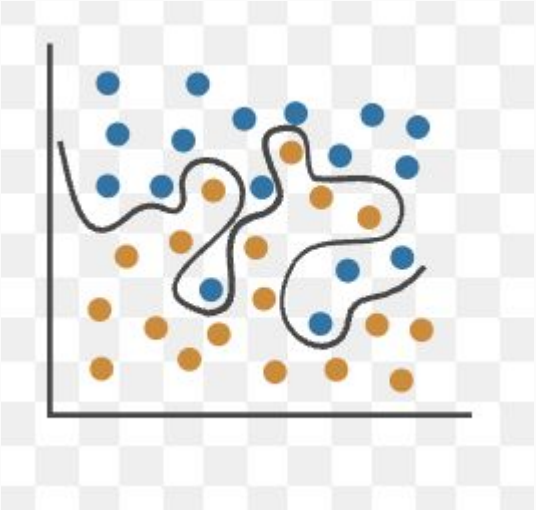
```python
model = tf.keras.Sequential([
    tf.keras.layers.Dense(
        128,
        activation='relu',
        kernel_initializer='he_normal'   # He initialization
    )
])
```

SOLVING OVERFITTING

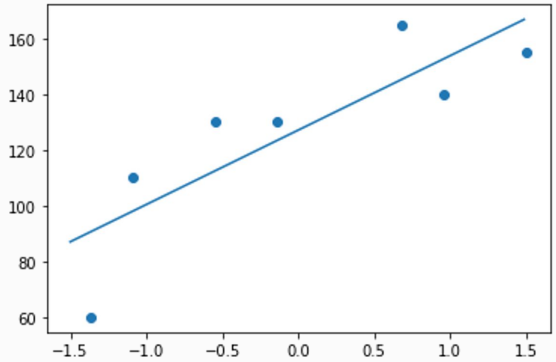
Overfitting in Neural Networks

Overfitting = memorizing training data, failing on new data.

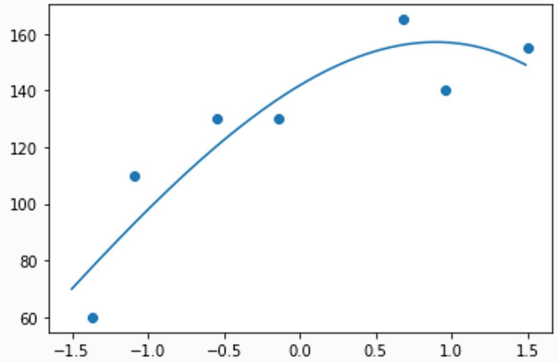
- Model learns **training data too well**, including noise.
- appears **highly accurate on training data** but performs poorly on unseen/test data.
- Example: Using very high-order polynomials in regression "memorizes" points instead of capturing general trends
    - When you use **very high-order polynomials** (say, 10th or 20th order)
        - can bend and twist curve to pass **exactly through every training point**.
        - fits training points perfectly
        - **oscillates wildly between them**, producing strange predictions for new inputs.



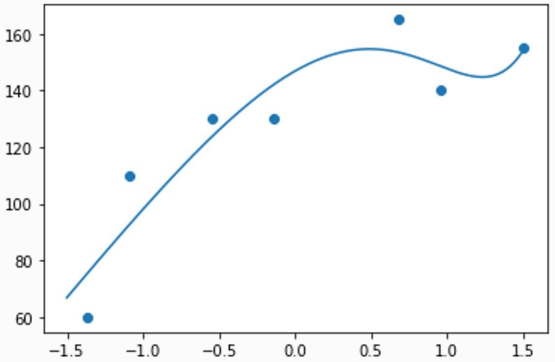
Overfitting Example



Few Parameters - Reasonable Fit



More Parameters - Better Fit



Even More Parameters - Going far inf fitting data

How can we avoid Overfitting

Causes of Overfitting:

- Too many features relative to data.
- Very complex network (many layers/neurons)
- Insufficient data or improper training.

Remedies

1. Regularization (L1/L2)
2. Early Stopping
3. Dropout
4. Good initialization
5. Proper data management

Monitoring **training vs validation performance** is crucial.

Method 1: Regularization

- model with **many parameters** → some parameters can become **very large** to "force" model to fit training data exactly.
  - In deep learning, or neural networks, model with many parameters means a deep or wide network
- Regularization reduces the magnitude of the parameters so that no single weight dominates the predictions.

$$\frac{\lambda}{2}\sum_{k}\theta_k^2$$

Training_Cost = Actual_Error + Penalty_for_High_Weights

e.g., Sum of Squared Error or Cross Entropy Loss Function

λ = regularization strength (controls penalty)

- Add a **penalty** for **large weights** in the cost function.
- Encourages the model to use **only necessary** parameters.

Regularization

L2-Regularization (Ridge)

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\text{Loss}(h_\theta(x^{(i)}), y^{(i)}) \; + \; \frac{\lambda}{2}\sum_{j=1}^{n}\theta_j^2$$

- Penalizes by the square of each weight to the cost.
- Weights shrink but mostly stay nonzero.

- m = number of training examples
- n = number of parameters (weights)
- θ_j = weight for feature j
- λ = regularization strength (controls penalty)
- Sum usually excludes θ_0 (bias term)

L1-Regularization (Lasso)

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\text{Loss}(h_\theta(x^{(i)}), y^{(i)}) \; + \; \lambda\sum_{j=1}^{n}|\theta_j|$$

- Penalizes by the absolute value of weights
- can shrink some weights to exactly zero, effectively "removing" them from the model.

Takeaway:

- L1 encourages **sparse models** (some features removed).
- L2 encourages **small, distributed weights** (all features kept but smaller).

Regularization

When to Use L2 (Ridge)

- Most common choice for **deep learning**.
- Works well when **all input features are potentially useful**.
- Helps **stabilize training**, especially in **deep networks with many parameters**.
- Prevents weights from becoming too large → reduces overfitting without completely ignoring features.
- Good **default option** for hidden layers in deep nets.

When to Use L1 (Lasso)

- Use when you want **sparsity**: automatically ignore irrelevant features.
- Useful for **feature selection** in shallow networks or small datasets.
- Helps make models **simpler and more interpretable**.
- Often combined with L2 (Elastic Net) if you want **both sparsity + stable training**.

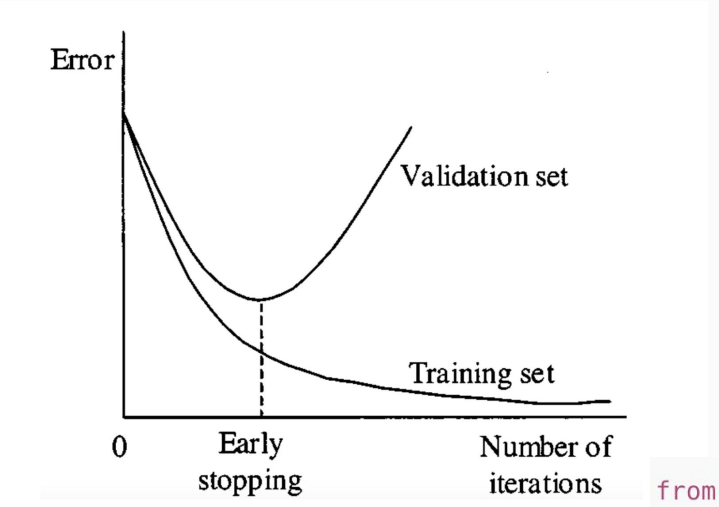
$$J(\theta) = \text{Loss} + \lambda_1 \sum_j |\theta_j| + \lambda_2 \sum_j \theta_j^2$$

Elastic Net Regularization

```
from tensorflow.keras import layers, regularizers

layer = layers.Dense(64, activation='relu',
                     kernel_regularizer=regularizers.l2(0.01))
```

Method 2: Early Stopping



- stopping the training process before model starts to overfit.
- idea
  - monitor model's performance on a validation set during training
  - stop training when performance starts to degrade, which is an indication that model is beginning to overfit training data.

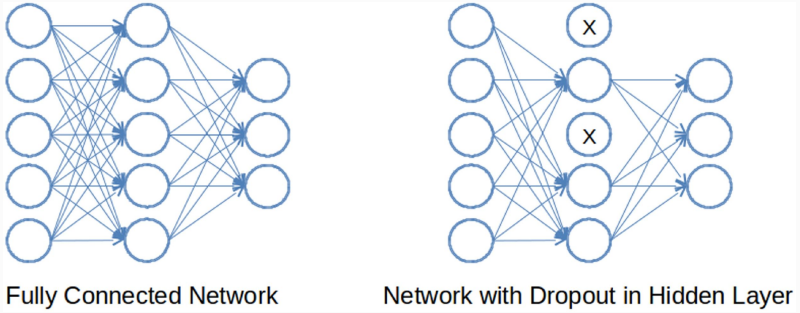
```
from tensorflow.keras.callbacks import EarlyStopping

es = EarlyStopping(monitor='val_loss', mode='min', patience=20)

model.fit(X_train, Y_train,
          validation_data=(X_val, Y_val),
          epochs=100,
          callbacks=[es])
```

Method 3: Dropout

- Some neurons develop **very high activations** and dominate network's decisions.
- This often happens when network **overfits to training data**, relying too heavily on a few specific neurons.

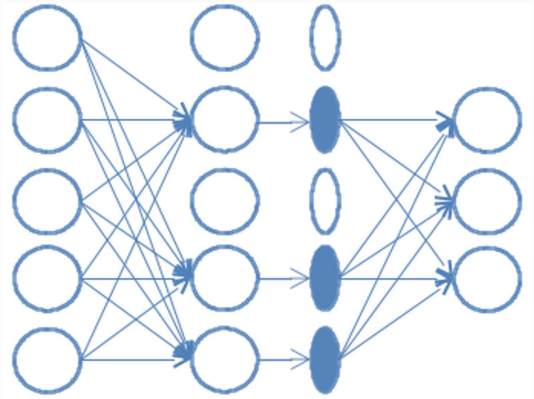


- Dropout
  - Idea is to randomly **turn off neurons** during training.
  - Prevents some neurons from **dominating or co-adapting** with others.
  - It's about **which neurons are active**, not their weight magnitude!

L1/L2 → "control the size of connections"
Dropout → "control reliance on specific neurons"

Drop out implemented via Mask

1. After computing activations for a hidden layer, create a **binary mask** of the same shape as the layer.
    - Each element = 0 (drop neuron) or 1 (keep neuron).
2. Multiply the hidden layer's activations by this mask.
    - Neurons with 0 → do not contribute to the next layer.
    - Neurons with 1 → active as usual.
3. Backpropagation only updates **active neurons**.



Dropout Implemented with Mask

randomly mask (turn off) *different neurons at every training iteration*

Other Remedies for Overfitting

4. Good Initialization

- Start weights so activations don’t explode.
- Prevents early layers from dominating and helps smoother learning.

5. Proper Data Management

- More data → network can learn general trends rather than memorizing.
- Techniques:
  - Data augmentation (images, text, etc.)
  - Proper train/validation/test splits
  - Shuffle data to avoid bias

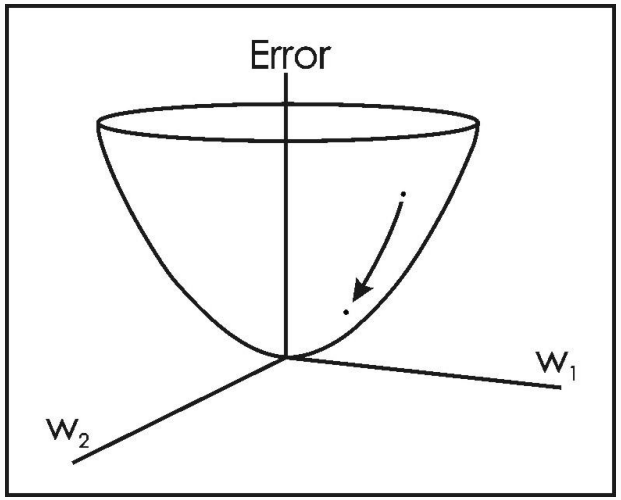
OPTIMIZATION METHODS

Optimization Methods

- Optimization: Adjust weights to minimize error efficiently

- Training deep neural networks involves adjusting millions of weights.
  - Each weight is updated to reduce error between predicted and actual output.
  - Choosing how much to adjust (learning rate) and how to update is critical.

- Goal is to efficiently reach a good solution without getting stuck or overshooting the minimum.



Gradient Descent Recap

Vanilla Gradient Descent:

$$\theta_j = \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

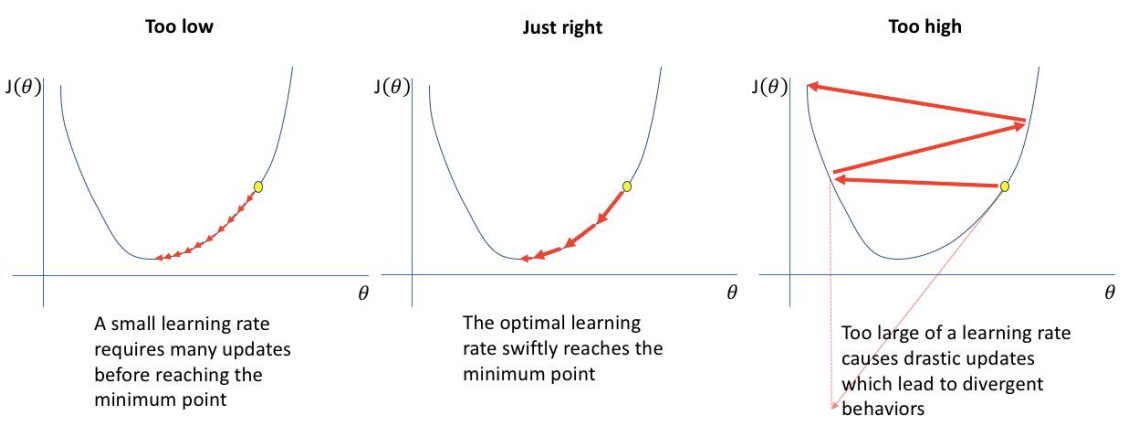
- θ_j = weight
- α = learning rate
- $J(\theta)$ = cost function

Intuition: Move step by step down the slope of the error surface.

Problems with Gradient Descent

1. **Learning Rate (α) Selection**
    - Too small → very slow learning
    - Too large → overshoot or divergence

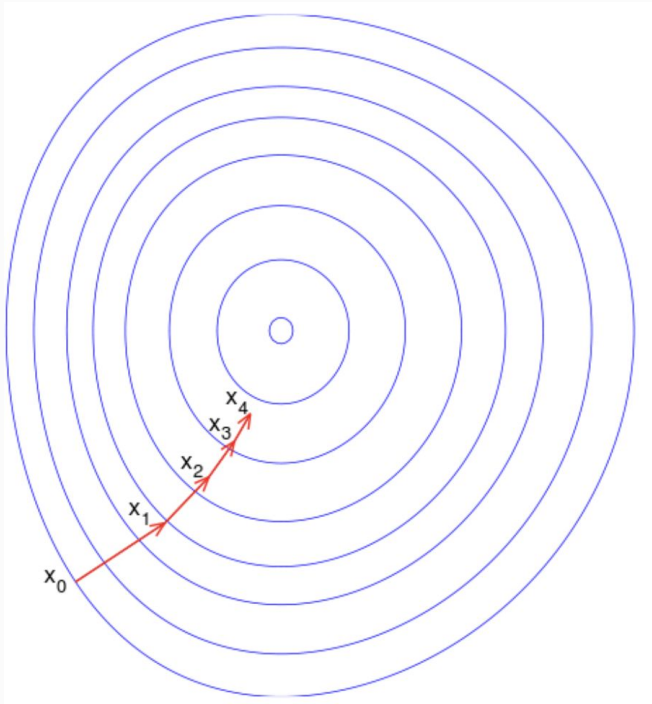
- Choosing the right value is difficult but critical to guide the learning well
- Grid search can help, but:
    - Computationally expensive
    - Time-consuming



Problems with Full Gradient Descent

2. All weights updated equally even if some need smaller or larger updates

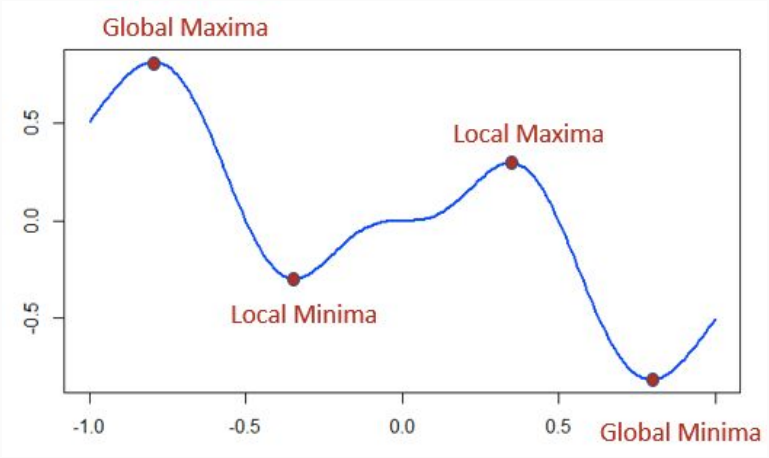
- Learning Rate Should Change Over Time
  - Early training:
    - We are far from the minimum
    - Prefer a larger $\alpha \rightarrow$ bigger steps
  - Later training:
    - Close to the minimum
    - Prefer a smaller $\alpha \rightarrow$ fine adjustments



Problems with Gradient Descent

3. Local Minima

- Standard GD can get stuck in local minima
- Complex neural network surfaces have many such points



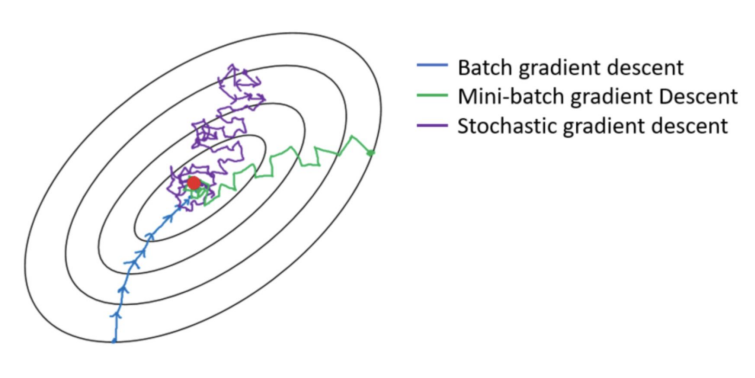
SOLUTIONS

We saw "vanilla GD"

Will see:

- Stochastic GD
- Adagrad
- Adam

1. Stochastic & Mini-Batch Gradient Descent



Idea: Update weights per sample or small batch rather than full dataset.

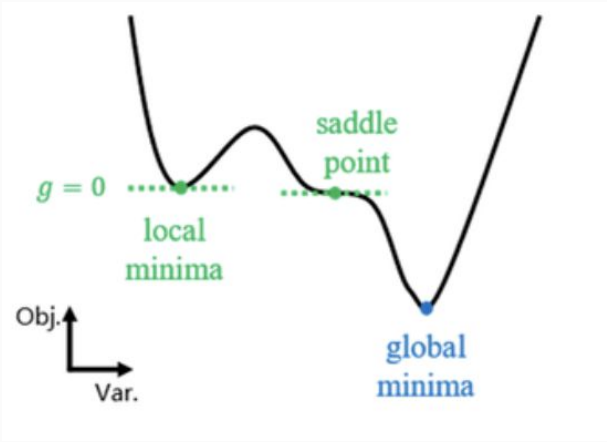
- SGD (Stochastic GD)
  - Update per single training example
  - Quite Expensive
- Mini-batch GD:
  - Update per batch

Intuition: Randomness helps explore the error surface better.

Note: Pure batch size = 1 is uncommon. When people say: “Use SGD”, they almost always mean: “Use mini-batch SGD.”

Why Gradient Descent cannot escape Local Minima or Saddle Points?

Local Minima and Saddle Points



Local Minima

- Full Gradient Descent updates using $\theta=\theta-\alpha\nabla J(\theta)$
- Now imagine you reach a **local minimum**, at that point: $\nabla J(\theta)=0$
    - That means:
        - The slope is zero.
        - There is no direction of descent.
        - The update becomes:
        - $\theta=\theta \rightarrow$ no update to the parameters, the algorithm stops.
- Because full batch GD is **deterministic**, it will stay there forever.
- It has **no randomness** to push it out.

Saddle Point

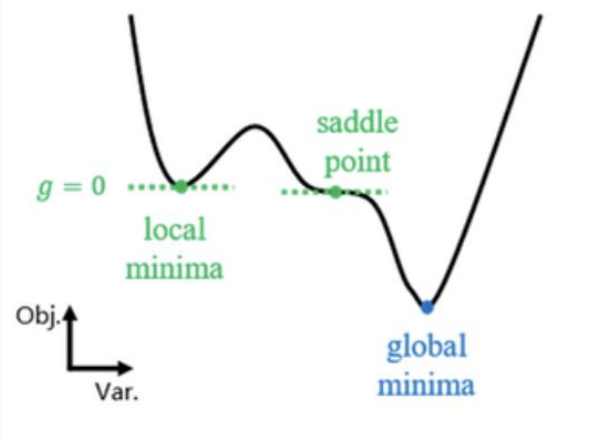
- Gradient = 0
- But it is NOT a minimum

How SGD can Escape?

- gradient is computed on a batch:
  - even if the true full gradient = 0, individual sample gradients are usually NOT exactly zero.
- This creates:
  - Small random pushes, tiny fluctuations
  - Movement away from the stationary point

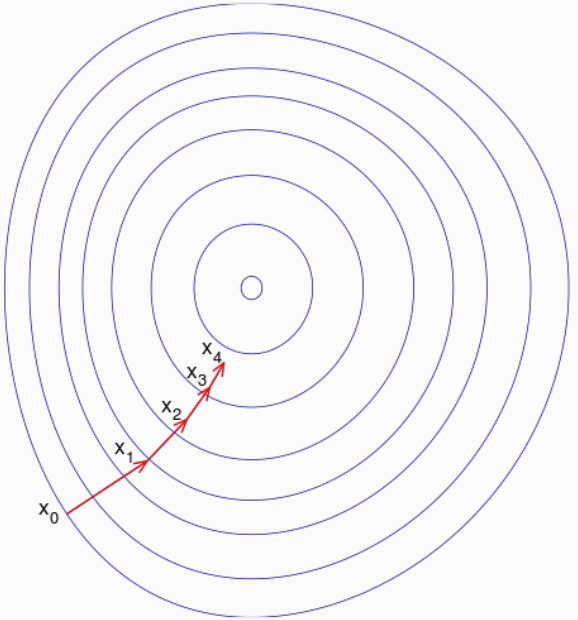
In deep neural networks:

- True bad local minima are rare.
- Saddle points are the real problem.
- High-dimensional spaces contain many flat saddle regions.
- SGD helps escape these flat areas.



Adaptive Learning Rates

- SGD improved vanilla GD.
- But a major breakthrough came from:
  - Using different learning rates for different parameters.
  - Updates depending on previous updates



2. Adagrad

- Adaptive learning rate per parameter
- Weights that receive large gradients frequently get smaller future updates.
- Weights that receive small gradients → get relatively larger updates.

- Frequently updated weights slow down.
- Rarely updated weights speed up.

$$\Delta\theta_j = -\eta \frac{g_j}{\sqrt{\sum_{\tau=1}^{t} g_{\tau,j}^2}}$$

Where:

- η = base learning rate
- $g_j = \frac{\partial J(\theta)}{\partial \theta_j}$

3. Adam Optimizer (Adaptive Moment Estimation)

- updates depend partly on previous updates
  - Normally each step depends only on the slope right now
  - With momentum, each step depends on the slope now and the direction we were already moving.
- Maintains two moving averages for each parameter:
  - Mean of gradients → helps with momentum (direction of previous updates)
  - Mean of squared gradients → helps scale updates based on past magnitudes

You can think of:
Adam = Momentum + Adaptive Learning Rate

Adam is popular:

- Fast convergence → learns quickly on most problems
- Works well out-of-the-box → minimal tuning needed
- Good default choice → often the first optimizer to try

Optimizers – Main vs. Variations

- Main Adaptive Optimizers
  - Adam – Combines momentum + RMSProp; adaptive learning rates
  - AdaGrad – Per-parameter adaptive learning rates; accumulates squared gradients
  - RMSProp – Like AdaGrad but uses moving average of squared gradients
- Variations / Enhancements
  - SGD + Momentum – Adds momentum to standard SGD to smooth updates
  - SGD with Nesterov – Momentum with gradient lookahead for faster convergence
  - Nadam – RMSProp + momentum + Nesterov lookahead
- Analytical / Less Common
  - L-BFGS – Quasi-Newton method using approximate Hessian; rarely used for large neural networks

Takeaways:

- Adam, RMSProp, AdaGrad are variations on adaptive learning rates.
- Momentum is often added to improve convergence.
- L-BFGS is mostly educational, not practical for large NNs.

Choosing an Optimizer - Practical Tip

- Try **Adam first**, it usually works well across different dataset sizes and architectures.
- Experiment with alternatives if results are unstable or the architecture is specialized.

Announcements

- *First Assessment is next week - on 11th of March*
  - You should be in the class for the assessment.
  - Everyone should be on Brightspace
- *Please try the mock quiz*
  - *Pass: MockQuiz*

Lab Work

- We have 2 notebooks this week, one for improving the performance through trying different activation functions and second for optimizers
- Run each and check the cells

APPENDIX